TEST PLAN

Combined Test Plan — Automation & Manual Testing Projects

Project 1	Lulu Mall, Bengaluru — Website Automation Testing
Project 2	Kothamangalam Agro Tourism (EchoTourism) — Website Manual Testing
Prepared By	[Test Lead / Sr. Test Engineer Name]
Document Version	1.0 (Draft)
Date	28-Sep-2026
Status	Draft — pending review & sign-off

This document drives all future testing activities for both projects. Fields in brackets [ ] are placeholders — fill these in with actual names, dates and estimates before circulating for sign-off.

1. Objective

To plan, guide and track all testing activities for two website QA efforts under one document: an automated Selenium/TestNG regression suite for the **Lulu Mall, Bengaluru** website, and a manual, test-case-driven verification effort for the **Kothamangalam Agro Tourism (EchoTourism)** website. The plan ensures both efforts are executed, tracked and signed off consistently.

#	Application	URL	Test Type
1	Lulu Mall, Bengaluru	www.bengaluru.lulumall.in	Automated (Selenium + TestNG + Java)
2	Kothamangalam Agro Tourism	kothamangalamagrotourism.in	Manual (test-case sheet, Google Sheets)

2. Effort Estimation

Indicative effort based on current test inventory (185 automated test classes / 264 @Test methods for Project 1; 16+ manual test cases logged so far for the STAY tab alone in Project 2, with more tabs to size). Update person-days once full scope per tab/module is confirmed.

Activity	Project	Estimated Effort
Test case/script review & planning	Both	[X] person-days
Automation script execution (full suite, 185 classes)	Project 1	[X] person-days
Automation script maintenance/fixes	Project 1	[X] person-days
Manual execution — remaining tabs (Header, Footer, Home, Experience, Contact Us)	Project 2	[X] person-days
Defect logging, triage & retest	Both	[X] person-days
Test summary reporting & sign-off	Both	[X] person-days

Effort figures are placeholders — size these using historical velocity or a 3-point (best/likely/worst) estimate per module before finalising the schedule in section 9.

3. Scope

In Scope

- Functional UI testing of both websites — navigation, content display, interactive elements (buttons, sliders, popups, galleries) and links/redirects.
- **Project 1** — all 8 modules currently automated: Home, Shops, Dine, ContactUS, Events, Entertainment, Offer, Location.
- **Project 2** — all 6 site sections tracked in the manual sheet: Header, Footer, Home, Experience, Stay, Contact Us.

Out of Scope

- Performance/load testing, security testing, and accessibility (WCAG) auditing.
- Backend/admin CMS functionality for either site.
- Real payment or booking transaction processing (only the trigger action, e.g. "Reserve your stay", is verified).
- Native mobile app testing — responsive web only.

4. Approach

Project 1 — Automation (Lulu Mall Bengaluru)

- Java + Selenium WebDriver + TestNG + ChromeDriver.
- One test class per UI element/interaction (e.g. *EventImg*, *Toggle*, *Slideclick*), grouped into packages by site module.
- Pattern: *@BeforeMethod* navigates to the page and scrolls to the target element; *@Test* methods (with *priority*) perform the interaction and verify via explicit waits.

Project 2 — Manual (Kothamangalam Agro Tourism)

- Scripted testing from a documented test case sheet — steps, action, expected result recorded per case.
- Testers work tab-by-tab, logging actual result and Pass/Fail status per row as they execute.

5. Assumption

- The target websites remain accessible and stable in their current (production) state throughout testing.
- Test environment (browser, OS) and access credentials, if any, are available before execution begins.
- Site content used as test data will not change significantly mid-cycle; if it does, locators and expected results will be re-baselined.
- The current automation project (Java/Selenium/TestNG) compiles and runs with a compatible ChromeDriver version for the tester's Chrome installation.
- Testers executing the manual sheet have access to edit the shared Google Sheets workbook.

6. Risk

Risk	Impact
Automation relies on hard-coded *Thread.sleep()* calls instead of explicit waits	Flaky/slow test runs, intermittent failures
No shared Page Object/base class — logic (footer, nav bar, social links) duplicated across ~185 classes	High maintenance cost as the site changes
No visible build tool (Maven/Gradle) or CI pipeline in the automation project	Hard to run consistently or automate execution
Manual test evidence lives only in a shared Google Sheet	Risk of accidental edits/loss, no formal defect linkage
Live production site used as the test target for both projects	Tests may break when real content/layout changes

7. Backup Plan / Mitigation Plan

- Migrate automation waits from *Thread.sleep()* to explicit *WebDriverWait* conditions throughout.
- Refactor common actions (footer, nav bar, social links, privacy/terms) into shared reusable classes (Page Object Model) to cut duplication.
- Introduce a build tool (Maven/Gradle) and wire the suite into a CI pipeline (e.g. GitHub Actions/Jenkins) so runs are repeatable and don't depend on one machine.
- Keep a version-locked copy of the manual test sheet at the start of each cycle, and link logged Fails to a lightweight issue tracker for traceability.

- Re-baseline locators and expected results promptly whenever site content or layout changes are announced.

8. Testing Methodology

- **Functional (black-box) testing** — verifying UI behaviour against expected results, with no knowledge of underlying implementation.
- **Project 1:** automated regression testing — repeatable scripted checks re-run on demand across all 8 modules.
- **Project 2:** scripted manual testing with an exploratory-informed approach — testers follow the documented steps but may probe adjacent behaviour when a Fail is observed.

9. Schedule

Indicative milestones — update dates to match the actual release calendar:

Milestone	Target Date	Owner
Test plan review & sign-off	[DD-MMM-YYYY]	[Test Lead]
Automation suite execution (full regression, 185 classes)	[DD-MMM-YYYY]	[Automation Engineer]
Manual test case execution — all 6 tabs	[DD-MMM-YYYY]	[Manual Tester]
Defect triage & retest	[DD-MMM-YYYY]	[QA Team]
Test summary report & sign-off	[DD-MMM-YYYY]	[Test Lead]

10. Test Environment

Item	Project 1 (Automation)	Project 2 (Manual)
Browsers	Google Chrome (primary) — [add Firefox/Edge if required]	[Chrome/Edge — confirm tester default]
OS	[Windows/macOS — confirm]	[Windows/macOS — confirm]
Tools	Selenium WebDriver, TestNG, Java, ChromeDriver	Google Sheets
Test data	Live production content on target site	Live production content on target site
Devices	Desktop (resolution [TBD])	Desktop (resolution [TBD])

11. Defect Tracking

Every Fail is logged as a defect with steps to reproduce, expected vs. actual result, severity, and screenshot/evidence, then tracked through Open → Fixed → Retest → Closed.

Currently open (Project 2, STAY tab):

Test Case	Description	Expected	Actual	Status
TC_KOTGMSTY_13	Click "Reserve your stay" button	Opens a new window	No new window opened	FAIL
TC_KOTGMSTY_15	Click icons under Farm Stay Amenities	New window each time	New window not opened	FAIL

Test Case	Description	Expected	Actual	Status
TC_KOTGMSTY_16	Click text under amenity icons	New window each time	New window not opened	FAIL

12. Test Automation

Test automation is scoped to Project 1 (Lulu Mall Bengaluru) only; Project 2 remains fully manual.

Module (Package)	Test Classes	Example Coverage
Home	34	Banners, homepage navigation, event image carousel
Shops	31	Shop directory, anchor store list, pagination, slider
Dine	29	Restaurant/cafe listings, category sections, cafe views
ContactUS	27	Contact form, address, click-to-call, token creation
Events	24	Event listing, dates, headings, links, add-to-calendar
Entertainment	20	Cinepolis section, footer/menu clicks, social links
Offer	18	Offers listing, gallery slider, footer content
Location	1	Landing/location page load

Total: 8 modules, 185 test classes, 264 @Test methods (current project export).

13. Deliverables

Deliverable	Project
This Test Plan document	Both
Automated test scripts (Java/TestNG source)	Project 1
Automation execution/test reports	Project 1
Manual test case register (Google Sheet, per-tab)	Project 2
Defect log / bug reports	Both
Test summary report (end of cycle)	Both

14. Entry and Exit Criteria

Entry Criteria

- Target site content is deployed to a stable, reachable environment.
- Automation: project compiles and referenced locators are confirmed present (smoke check).
- Manual: test case sheet for the section under test is reviewed and free of blocking ambiguities.
- Required test data and access credentials are available.

Exit Criteria

- 100% of planned test cases (automated and manual) have been executed.
- No open Critical or High severity defects remain unresolved.
- Pass rate meets the agreed threshold (recommended ≥ 95% — team to confirm).

- All defects are logged, retested where fixed, and closed.

15. Test Stop Criteria

- **Suspend testing** if the site under test is unreachable, a blocking defect prevents further navigation, or tooling issues (e.g. ChromeDriver version mismatch) block automation execution.
- **Resume testing** once the blocking issue is resolved and confirmed via a quick smoke check of the affected module.

16. Role and Responsibilities

Role	Responsibility
Test Lead / Sr. Test Engineer	Owns the test plan, reviews results, approves release sign-off
Automation Engineer	Maintains and executes the Selenium/TestNG suite, triages script failures
Manual Tester	Executes manual test cases, updates the Google Sheet, logs defects
Developer(s)	Fixes defects, supports root-cause analysis, confirms fixes for retest

17. Templates

Manual Test Case Template (Project 2)

Test Case ID	Description	Steps	Action	Expected Result	Actual Result	Status
TC_KOTGMSTY_13	Verify user can click Reserve your stay button	1. Navigate to stay.php 2. Click the Reserve your stay button	Clicking	User should get a new window	Not as expected	FAIL

Automation Test Class Template (Project 1)

Naming convention: **<ActionOrElement>.java** inside the package for its module (e.g. *Home/EventImg.java*). Standard structure per class:

- **@BeforeMethod** — launch driver, navigate to page, maximize window, scroll to target element.
- **@Test(priority = n, description = "...")** — perform the click/interaction, wait for the expected condition, assert the result.
- Driver is closed at the end of the test method.

Defect Report Template

Field	Example
Defect ID	[DEF-001]
Test Case ID	TC_KOTGMSTY_13
Summary	Reserve your stay button does not open a new window
Severity / Priority	[Medium / High]
Steps to Reproduce	1. Navigate to stay.php 2. Click "Reserve your stay"
Expected vs Actual	New window expected; no window opened
Status	Open / Fixed / Retest / Closed